

Lab Practice Exercises 1

Lecture 1 · Practice only — nothing here is collected or graded

Fall Semester 2026 · Dr. Narasimha Rao Vajjhala · nrao@aubg.edu · Office MB 253

BEFORE YOU START

Environment	JDK 25 (the current Long-Term Support release) and IntelliJ IDEA. Check your setup with <code>javac -version</code> and <code>java -version</code> before Question 1.
Language form	You may write your answers either as a classic <code>public class</code> with a <code>static void main</code> , or as a Java 25 compact source file with an instance <code>void main()</code> . Question 22 asks you to produce both.
Style	Get into the habit now of naming things properly, indenting consistently and commenting as you go. It costs nothing here, and it is part of the mark on the assignments and the project.
Nothing to hand in	These exercises are for practice. There is nothing to submit, no deadline and no mark — keep your <code>.java</code> files somewhere you can find them again, and bring anything that will not work to the lab or to office hours.

Questions 1 to 10 are the core set and cover output, keyboard input, arithmetic and casting. Questions 11 to 22 cover the rest of Lecture 1 — the toolchain, program anatomy, comments and style, the three kinds of error — and extend the data-type material a little further.

None of this is collected, so work at whatever pace suits you and skip anything you already find easy. Two suggestions all the same. Type the programs out rather than pasting them — the mistakes you make while typing are the ones worth having. And do not skip the questions that ask you to break something on purpose: 12, 15, 16 and 18 are the ones that pay off later, because you will meet those same errors again when they are not deliberate. The material is examinable even though the exercises are not.

Part A · Core Exercises

Question 1 · Introduction Card

Write a Java program that prints a three-line introduction card about yourself. The program should display your name, your major, and one short fun fact.

EXPECTED OUTPUT (EXAMPLE)

```
Name: John Student
Major: Computer Science
Fun fact: I can solve a Rubik's cube in under a minute.
```

Question 2 · Personalised Welcome

Write a Java program that asks the user for their name and then prints a personalized welcome message.

EXPECTED OUTPUT (EXAMPLE)

```
Enter your name: John
Welcome to Java programming, John!
```

Question 3 · Circle Pizza Area Calculator

Write a program that asks the user to enter the radius of a pizza in centimeters, computes the area using a constant value for PI, and displays the result.

EXPECTED OUTPUT (EXAMPLE)

```
Enter radius (cm): 10
Pizza area is 314.159 cm^2
```

Question 4 · Change Maker

Write a Java program that reads a money amount as a double value and displays the maximum number of dollars, quarters, dimes, nickels, and pennies.

EXPECTED OUTPUT (EXAMPLE)

```
Enter amount: 11.56
Dollars: 11
Quarters: 2
Dimes: 0
Nickels: 1
Pennies: 1
```

Question 5 · Seconds to Hours, Minutes and Seconds

Write a Java program that reads a number of seconds and converts it into hours, minutes, and seconds.

EXPECTED OUTPUT (EXAMPLE)

```
Enter total seconds: 3665
3665 seconds = 1 hour(s), 1 minute(s), 5 second(s)
```

Question 6 · Day of the Week

Assume that days of the week are numbered from 0 to 6. Write a program that asks for today's day number and a number of days in the future, then displays the future day number.

EXPECTED OUTPUT (EXAMPLE)

```
Enter today's day: 6
Enter days in the future: 10
Future day is: 2
```

Question 7 · Loan Payment

Write a Java program that asks the user for the annual interest rate, number of years, and loan amount, then computes and displays the monthly payment and total payment.

EXPECTED OUTPUT (EXAMPLE)

```
Enter annual interest rate: 5.5
Enter number of years: 5
Enter loan amount: 10000
Monthly payment is: 191.01
Total payment is: 11460.70
```

Question 8 · Scientific Notation

Write a Java program that reads a double value entered in scientific notation and prints the equivalent decimal value.

EXPECTED OUTPUT (EXAMPLE)

```
Enter a number: 1.23e2
You entered: 123.0
```

Question 9 · Points Tracker

Write a Java program that starts with zero points and allows the user to enter three earned point values, updating the total using augmented assignment operators.

EXPECTED OUTPUT (EXAMPLE)

```
Enter points: 10
Enter points: 25
Enter points: 7
Final points: 42
```

Question 10 · Casting and Truncation

Write a Java program that reads a double value and displays the original value, the value cast to an integer, and the value truncated to two decimal places using casting.

EXPECTED OUTPUT (EXAMPLE)

```
Original: 3.987
As int: 3
Truncated to 2 decimals: 3.98
```

Part B · Additional Exercises

These questions cover the parts of Lecture 1 that Part A does not reach. Several of them ask you to record what the compiler or the runtime actually printed — copy those messages exactly, character for character, rather than paraphrasing them.

B1 · THE TOOLCHAIN: JAVAC, JAVA AND THE CLASS FILE

Question 11 · Compile and Run Without the IDE

Create a file named `Welcome.java` containing the program below. Then, working in a terminal rather than with the IDE's Run button, carry out the following steps and record the exact output of each one.

WELCOME.JAVA

```
// Q11 - the classic form, used for the command-line exercise.
public class Welcome {
    public static void main(String[] args) {
        System.out.println("Welcome to Java!");
    }
}
```

- (a) Confirm your toolchain is JDK 25 with `javac -version` and `java -version`.
- (b) Compile with `javac Welcome.java`, then list the directory. What new file appeared, and what is inside it?
- (c) Run the program with `java Welcome`.
- (d) Now run `java Welcome.class` and record the error exactly as printed.
- (e) Delete the class file and run `java Welcome.java` instead.

Finish with two or three sentences explaining what `javac` produced in step (b), why step (d) fails, and what the single-file source launcher in step (e) does differently from steps (b) and (c) taken together.

Question 12 · The File-Name Rule

Save the program below in a file called `Greeting.java` — note that the file name and the class name do not agree — and try to compile it.

GREETING.JAVA

```
public class Hello {
    public static void main(String[] args) {
        System.out.println("Hello from COS 2030");
    }
}
```

Copy the compiler's message word for word. Then fix the problem in two different ways: first by renaming the file, and then, starting again from the original, by editing the source instead.

In one sentence, say which of the two fixes you would use in a real project, and why.

B2 · ANATOMY, COMMENTS AND STYLE

Question 13 · A Style Pass

The program below compiles and prints correct numbers, but no one could be asked to maintain it. Rewrite it so that it follows the conventions from Lecture 1.

THE PROGRAM AS IT STANDS

```
public class messyrectangle{public static void main(String[] a){double X=12.5;double y=4;
double Z=X*y;double p=2*(X+y);System.out.println("area"+Z);
    System.out.println("perimeter"+p);
double SCALE=2.0;double z2=(X*SCALE)*(y*SCALE);System.out.println("scaled area"+z2);}}
```

Your rewrite must use UpperCamelCase for the class, lowerCamelCase for the variables, and UPPER_SNAKE_CASE for the scale factor, which should be `final`. Put one statement on each line, indent every nested block by four spaces, and leave a single space on each side of every binary operator.

Include all three comment forms: a Javadoc comment above the class saying what the program does, a block comment explaining the scaled-area calculation, and at least one end-of-line comment. Finally, label the three numbers in the output so a reader can tell them apart.

Question 14 · print, println and Escape Sequences

Write a program that prints the receipt below exactly as shown. Use `System.out.print` at least once so that two pieces of output share a line, and use the escape sequences `\t`, `\n`, `\` and `\\` where they are needed. Line the columns up with tab characters, not with runs of spaces.

EXPECTED OUTPUT (EXAMPLE)

```
+-----+
|      AUBG COFFEE BAR      |
+-----+
Item           Qty      Price
Espresso       2        3.00
Croissant      1        2.50

Cashier said: "Thank you!"
Saved to: C:\receipts\2026\september.txt
A backslash on its own: \
A tab between  these two words.
```

B3 · THE THREE KINDS OF ERROR

Question 15 · Syntax Errors

The file below contains five separate defects. Compile it and you will find that the compiler does not report all five at once.

BROKENGREETING.JAVA

```
public class BrokenGreeting {
    public static main(String[] args) {
        String course = "COS 2030"
        int Year = 2026;
        System.out.println("Welcome to " + course);
        System.out.println("Academic year: " + year);
        System.out.println("Have a good semester!")
    }
}
```

Fix the defects the compiler names, recompile, and repeat until the file compiles cleanly. Record the messages you received at each pass, and give the corrected program.

Then answer this: why did the fifth defect stay hidden until the first four had been repaired? What does that tell you about how to read a long list of compiler errors?

Question 16 · Runtime Errors

A program that compiles can still fail while it is running. The program below does exactly that.

RUNTIMECRASH.JAVA

```
public class RuntimeCrash {
    public static void main(String[] args) {
        int[] scores = { 90, 75, 88 };
        int count = 0;
        System.out.println(100 / count);
        System.out.println(scores[3]);
    }
}
```

(a) Compile and run it. Copy the exception name, its message and the line number from the stack trace, then explain in one sentence what the program asked the computer to do that it could not do.

(b) Only the first failure is reported. Comment that line out, run the program again, and record the second failure the same way.

(c) Write your own program that triggers both failures deliberately but catches each one, so that the program reaches its last line and reports both. In the same program print `100.0 / 0` and `0.0 / 0.0`, and explain in one sentence why those two do not throw an exception even though dividing an `int` by zero does.

Question 17 · Logic Errors

Neither half of the program below fails to compile, and neither crashes. Both print the wrong answer — which is what makes logic errors the expensive kind.

LOGICERRORS.JAVA

```
public class LogicErrors {
    public static void main(String[] args) {
        // Program A - convert 35 degrees Celsius to Fahrenheit. Expected: 95
        System.out.print("35 C in Fahrenheit is ");
        System.out.println((9 / 5) * 35 + 32);

        // Program B - the average of three test scores. Expected: 84.33
        int a = 90, b = 75, c = 88;
        System.out.println("Average: " + (a + b + c) / 3);
    }
}
```

For each of the two parts, state what the program prints, what it should print, the single language rule that explains the difference, and the smallest change that fixes it. Then give the corrected program with both fixes applied.

B4 · DATA TYPES AND EXPRESSIONS, CONTINUED

Question 18 · Named Constants

Write a price calculator in which no numeric literal appears more than once. The program reads a list price, applies a 10% student discount, adds 20% VAT to the discounted figure, and prints the four lines shown below with the amounts right-aligned in a field eight characters wide.

EXPECTED OUTPUT (EXAMPLE)

```
Enter list price: 49.99
List price:      49.99 EUR
```

```
After discount:    44.99 EUR
VAT:              9.00 EUR
Total:            53.99 EUR
```

Declare the discount rate, the VAT rate and the currency symbol as `final` constants named in `UPPER_SNAKE_CASE`.

Then add one line that assigns a new value to one of those constants and try to compile. Note the error the compiler gives you, and say at which of the three stages — compile time, run time, or neither — the problem was caught. Comment the line out again afterwards so the program still runs.

Question 19 · Formatted Output with `printf`

Print the small table below twice: first by building each line with string concatenation, and then with `System.out.printf`.

EXPECTED OUTPUT (EXAMPLE)

STUDENT	CREDITS	GPA
Ada	6	3.50
Alexandrina	18	3.88
Bo	120	4.00

The data is: Ada, 6 credits, GPA 3.5; Alexandrina, 18 credits, GPA 3.875; Bo, 120 credits, GPA 4.0.

In the second version use `%-14s` to left-align the name, `%6d` for the credit count and `%8.2f` for the GPA, and end each line with `\n` rather than `\n`. Add two further `printf` calls: one printing 87.5 as a percentage to one decimal place with a literal per-cent sign after it, and one printing 42 padded with leading zeros to five digits.

Finish with one sentence saying why the second table is easier to read than the first, and note what `%8.2f` did to Alexandrina's GPA of 3.875.

Question 20 · The char Type

Write a program that reads one letter from the keyboard and prints: the character itself; its Unicode code point as a decimal number; the same code in hexadecimal; the character that follows it in the alphabet; and the letter in upper case and in lower case.

EXPECTED OUTPUT (EXAMPLE)

```
Enter a single letter: y
Character: y
Unicode code point: 121
As hex: 79
Next character: z
Upper case: Y
Lower case: y
```

Obtain the following character by arithmetic and a cast rather than by a library call, and say why the cast is needed there when no cast was needed to turn the `char` into an `int`.

Then print the value of the expression `'a' - 'A'` and explain in one sentence what that number tells you about how upper-case and lower-case letters are stored.

Question 21 · Math Methods and the Limits of int

This question is in three short parts.

- (a) Print the value of `Math.pow(2, 10)`, `Math.sqrt(144)`, `Math.max(17, 42)`, `Math.abs(-8.25)`, `Math.round(3.5)`, `Math.round(-3.5)`, `Math.ceil(3.1)` and `Math.floor(3.9)`. Two of these results surprise most people the first time they see them. Say which two, and why.
- (b) Print `Integer.MAX_VALUE`, then print `Integer.MAX_VALUE + 1` and explain the result you get. Then print `Integer.MAX_VALUE + 1L` and explain why that one behaves differently.
- (c) Print `0.1 + 0.2`. Explain in one sentence why the answer is not `0.3`, and show one way to display it as `0.30`.

Question 22 · Back to the Classic Form

Java 25 lets you write a program as a compact source file: no class header, no `static`, and `IO.println` in place of `System.out.println`. It is convenient for a small exercise, but it is not the form used by your textbook, by existing Java code, or by the quizzes and the final exam.

Take your answer to Question 1 and write it in the classic, explicit form: a `public class` whose name matches the file name, a `public static void main(String[] args)` method, and `System.out.println`. Run both versions and confirm the output is identical.